

Reviewer Pack — Cheeger Constant Exponentiates Resolution Length for Tseitin...

Entry 933eeb640129 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 19:08:21 UTC

Cheeger Constant Exponentiates Resolution Length for Tseitin Formulas

Entry ID: 933eeb640129 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 19:08:21 UTC

1. Conjecture

Field A (mathematical branch): Spectral Geometry **Field B** (complexity object): Resolution Proof Length

Statement:

For a Tseitin formula derived from a d -regular graph G , the Resolution proof length is at least $2^{\Omega(h(G))}$, where $h(G)$ is the Cheeger constant of G .

Rationale (proposer's reasoning):

Expanders have high Cheeger constants, which correlate with the difficulty of Resolution proofs. The spectral properties of the graph influence the structure of the proof, leading to exponential lower bounds.

Taxonomy category: TSEITIN_RES (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7aef2e46ec3d4182

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (d * n) % 2 != 0:
            return None
        graph = {i: [] for i in range(n)}
        edges = set()
        for i in range(n):
            neighbors = random.sample(range(n), d - len(graph[i]))
            for neighbor in neighbors:
                if (i, neighbor) not in edges and (neighbor, i) not in edges:
                    graph[i].append(neighbor)
                    graph[neighbor].append(i)
                    edges.add((i, neighbor))
        return graph

    def cheeger_constant(graph):
        n = len(graph)
        min_cut = float('inf')
        for node in range(n):
            neighbors = graph[node]
            cut_size = len(neighbors)
            boundary_size = sum(1 for neighbor in neighbors if len(graph[neighbor]) > d)
            if boundary_size < min_cut:
                min_cut = boundary_size
        return min_cut / n

    def resolution_length(graph):
        # Simplified DPLL-based algorithm with pruning
        stack = []
        assignment = {}
        clauses = []
        for node in range(len(graph)):
            clause = [node, -node]
            clauses.append(clause)

        def dpll():
            if not clauses:
                return True
            unit_clause = next((c for c in clauses if len(c) == 1), None)
            if unit_clause:
```

```

        literal = unit_clause[0]
        assignment[literal] = True
        clauses = [c for c in clauses if literal not in c and -literal not in c]
        pure_literal = next((l for l in range(1, len(graph) + 1) if l not in assignment and -l not in assignment))
        if pure_literal:
            assignment[pure_literal] = True
            clauses = [c for c in clauses if pure_literal not in c and -pure_literal not in c]
        if not clauses:
            return True
        literal = random.choice([l for l in range(1, len(graph) + 1) if l not in assignment and -l not in assignment])
        assignment[literal] = True
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        if dpll():
            return True
        del assignment[literal]
        assignment[-literal] = True
        new_clauses.extend([c for c in clauses if literal in c or -literal in c])
        if dpll():
            return True
        del assignment[-literal]
        return False

    length = 0
    while not dpll():
        length += 1
    return length

n = random.randint(5, 40)
d = 2 * random.randint(1, n // 2)
graph = generate_d_regular_graph(n, d)
if graph is None:
    return {
        "metric_name": "Resolution Length",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

h_G = cheeger_constant(graph)
length = resolution_length(graph)

return {
    "metric_name": "Resolution Length",
    "metric_value": length,
    "instances_tested": 1,
    "conjecture_holds": length >= 2 ** (h_G * math.log(2)),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:

```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_length = sum(r["metric_value"] for r in results) / len(results)
std_length = math.sqrt(sum((r["metric_value"] - mean_length) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7536d2ac.py", line 116, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7536d2ac.py", line 100, in run_trial
    length = resolution_length(graph)
              ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7536d2ac.py", line 83, in resolution_length
    while not dpll():
              ^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7536d2ac.py", line 56, in dpll
    if not clauses:
          ^^^^^^^
UnboundLocalError: cannot access local variable 'clauses' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with UnboundLocalError, preventing data collection. The error suggests a code implementation flaw in the DPLL solver. | next: Fix the DPLL implementation to handle clause initialization properly and re-run the experiment

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110899
2	propose	ollama_remote	qwen3:8b	0	0	63842
3	preregistration	ollama_remote	qwen3:8b	0	0	24021
4	novelty	ollama_remote	qwen3:8b	0	0	20748
5	novelty	ollama_remote	qwen3:8b	0	0	16057
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16318
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12072
8	judge	ollama_remote	qwen3:8b	0	0	17210

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 281166 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/933eeb640129.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/933eeb640129.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/933eeb640129.tar.gz` (if generated)